

Nama : Rayya Syauqi Alulu'I / Jehezkiel Beny Brian
NIM : 103012300268 / 103012300241
Kelas : IF-46-DSIS.01 PCD

Catatan

- Karena tidak ada catetan untuk menggunakan maka tidak digunakan padding
- Karena tidak ada stride yang jelas maka diasumsikan stride = 1
- Citra Asli

Citra asli							
0	10	1	8	1	8	8	1
8	0	8	1	28	0	10	1
29	10	29	28	28	29	28	1
3	10	3	1	5	0	3	6
28	0	28	29	1	1	10	15
28	1	29	10	10	1	10	1
29	28	28	0	12	29	3	21
29	30	3	1	5	0	1	6

- Filter2:

Box filter

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

Weighted average filter

$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$
$\frac{2}{16}$	$\frac{4}{16}$	$\frac{2}{16}$
$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$

Mask/filter konvolusi

1	1	1
1	3	1
1	1	1

- Kode Digunakan

```
def conv2D(matrix:np.array,kernel:np.array):  
    # get the dimensions of the matrix and kernel:  
    m_rows, m_cols = matrix.shape  
    k_rows, k_cols = kernel.shape  
  
    # no padding (fill it with zeros), so the output dimensions will be:  
    out_rows = m_rows - k_rows + 1  
    out_cols = m_cols - k_cols + + 1  
  
    # initialize the output matrix:  
    output = np.zeros((out_rows+2, out_cols+2), dtype=np.uint8)  
  
    for i in range(out_rows):  
        for j in range(out_cols):  
            # extract the region of interest:  
            region = matrix[i:i+k_rows, j:j+k_cols]  
            # perform element-wise multiplication and sum:  
            output[i+1, j+1] = np.clip(np.round(np.sum(region * kernel))), 0, 255)  
  
    return output
```

✓ 0.0s

Kode diatas adalah kode untuk melakukan operasi konvolusi (tanpa padding). Pertama akan diambil ukuran matrix utama dan kernel untuk konvolusinya, dari situ buat matrix output (inisialisasi 0) lalu lakukan operasi konvolusi dengan kernel terhadap matrix utama.

- Matrix utama

```
original_matrix = np.array([  
    [0, 10, 1, 8, 1, 8, 8, 1],  
    [8, 0, 8, 1, 28, 0, 10, 1],  
    [29, 10, 29, 28, 28, 29, 28, 1],  
    [3, 10, 3, 1, 5, 0, 3, 6],  
    [28, 0, 28, 29, 1, 1, 10, 15],  
    [28, 1, 29, 10, 10, 1, 10, 1],  
    [29, 28, 28, 0, 12, 29, 3, 21],  
    [29, 30, 3, 1, 5, 0, 1, 6]  
])
```

✓ 0.0s

Gambar diatas adalah matrix utama yang akan di konvolusi.

- **Box Filter**

```
kernel_box = np.array([
    [1/9, 1/9, 1/9],
    [1/9, 1/9, 1/9],
    [1/9, 1/9, 1/9]
])
```

✓ 0.0s

```
# citra hasil menggunakan kernel box:
result_box = conv2D(original_matrix, kernel_box)
print("Hasil konvolusi dengan kernel box: (dibulatkan ke bilangan bulat terdekat)")
print(result_box)
```

✓ 0.0s

Menggunakan kode yang telah di definisikan diatas dan kernel yang telah terdapat dalam soal, didapatkan hasil citra hasil konvolusi seperti dibawah:

```
Hasil konvolusi dengan kernel box: (dibulatkan ke bilangan bulat terdekat)
[[ 0  0  0  0  0  0  0  0]
 [ 0 11 11 15 15 16 10  0]
 [ 0 11 10 15 13 15  9  0]
 [ 0 16 15 17 14 12 10  0]
 [ 0 14 12 13  6  5  5  0]
 [ 0 22 17 16 10  9 10  0]
 [ 0 23 14 11  8  8  8  0]
 [ 0  0  0  0  0  0  0  0]]
```

- **Weighted average filter**

```
weighted_kernel = np.array([
    [1/16, 2/16, 1/16],
    [2/16, 4/16, 2/16],
    [1/16, 2/16, 1/16]
])
```

✓ 0.0s

```
# citra hasil menggunakan kernel weighted:
result_weighted = conv2D(original_matrix, weighted_kernel)
print("\nHasil konvolusi dengan kernel weighted: (dibulatkan ke bilangan bulat terdekat)")
print(result_weighted)
```

✓ 0.0s

Masih menggunakan kode konvolusi diatas dengan kernel yang telah didefinisikan dalam soal, berikut adalah citra hasil konvolusi menggunakan weighted average filter;

```

Hasil konvolusi dengan kernel weighted: (dibulatkan ke bilangan bulat terdekat)
[[ 0  0  0  0  0  0  0  0]
 [ 0  8  9 13 15 13 10  0]
 [ 0 12 14 17 18 17 13  0]
 [ 0 12 13 14 10  9  9  0]
 [ 0 12 16 15  7  4  7  0]
 [ 0 18 19 15  9  8  8  0]
 [ 0 24 17  9  9 11  9  0]
 [ 0  0  0  0  0  0  0  0]]

```

- Mask/filter konvolusi

Mask/Filter konvolusi

```

mask_kernel = np.array([
    [1, 1, 1],
    [1, 3, 1],
    [1, 1, 1]
])

```

✓ 0.0s

```

result_mask = conv2D(original_matrix, mask_kernel)
print("\nHasil konvolusi dengan kernel mask: (dibulatkan ke bilangan bulat terdekat)")
print(result_mask)

```

✓ 0.0s

Masih menggunakan kode konvolusi diatas dengan kernel yang telah didefinisikan dalam soal, berikut adalah citra hasil konvolusi menggunakan mask/filter konvolusi:

```

Hasil konvolusi dengan kernel mask: (dibulatkan ke bilangan bulat terdekat)
[[ 0  0  0  0  0  0  0  0]
 [ 0 95 111 134 187 140 106  0]
 [ 0 120 148 187 176 189 134  0]
 [ 0 160 144 154 132 105  99  0]
 [ 0 130 167 174  60  43  67  0]
 [ 0 201 211 167 113  79 111  0]
 [ 0 255 186  98  92 129  78  0]
 [ 0  0  0  0  0  0  0  0]]

```

- Kode untuk minmax dan median filter

Citra menggunakan Max,min, median Filter (3x3)

```
def filter(matrix:np.array,filter_type:str):
    M,N = matrix.shape

    window = np.zeros(9)
    image_result = np.zeros((M-2,N-2))

    for i in range(1,M-1):
        for j in range(1,N-1):
            window[0] = matrix[i-1][j-1]
            window[1] = matrix[i-1][j]
            window[2] = matrix[i-1][j+1]
            window[3] = matrix[i][j-1]
            window[4] = matrix[i][j]
            window[5] = matrix[i][j+1]
            window[6] = matrix[i+1][j-1]
            window[7] = matrix[i+1][j]
            window[8] = matrix[i+1][j+1]

            if filter_type == "median":
                image_result[i-1][j-1] = np.median(window)
            elif filter_type == "max":
                image_result[i-1][j-1] = np.max(window)
            elif filter_type == "min":
                image_result[i-1][j-1] = np.min(window)

    return image_result
```

Kode diatas merupakan kode untuk menerapkan max,min dan median filter. Pertama diambil bentuk matrix yang akan dilakukan operasi, setelah itu buat window untuk menyimpan nilai2 dalam kernel. Lalu masukkan nilai2 dalam kernel ke dalam window, berdasarkan jenis filter (max, min, atau median) isi nilai pada image result dengan min, max, dan mediannya. Seperti operasi konvolusi kernel akan berpindah dengan stride = 1 (ukuran kernel adalah 3x3 dan tidak ada padding).

- Max filter

```
max_result = filter(original_matrix, "max")
# adding zero padding to the max_result to make it the same size as original_matrix
max_result = np.pad(max_result, pad_width=1, mode='constant', constant_values=0)
print("\nHasil filter max:")
print(max_result)
```

Gunakan function filter (kode diatas) dengan parameter max. Beri padding untuk mengoutputkan sesuai dengan ukuran input (karena asumsi tidak ada padding maka kasih nilai 0)

Hasil filter:

```

Hasil filter max:
[[ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0. 29. 29. 29. 29. 29. 29. 0.]
 [ 0. 29. 29. 29. 29. 29. 29. 0.]
 [ 0. 29. 29. 29. 29. 29. 29. 0.]
 [ 0. 29. 29. 29. 29. 10. 15. 0.]
 [ 0. 29. 29. 29. 29. 29. 29. 0.]
 [ 0. 30. 30. 29. 29. 29. 29. 0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.]]

```

- Median filter:

```

median_result = filter(original_matrix, "median")
# adding zero padding to the median_result to make it the same size as original_matrix
median_result = np.pad(median_result, pad_width=1, mode='constant', constant_values=0)
print("\nHasil filter median:")
print(median_result)
✓ 0.0s

```

Gunakan function filter (kode diatas) dengan parameter median. Beri padding untuk mengoutputkan sesuai dengan ukuran input (karena asumsi tidak ada padding maka kasih nilai 0)

Hasil filter:

```

Hasil filter median:
[[ 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  8.  8.  8.  8. 10.  8.  0.]
 [ 0.  8.  8.  8.  5. 10.  3.  0.]
 [ 0. 10. 10. 28.  5.  5.  6.  0.]
 [ 0. 10. 10. 10.  1.  3.  3.  0.]
 [ 0. 28. 28. 12. 10. 10. 10.  0.]
 [ 0. 28. 10. 10.  5.  5.  3.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.]]

```

- Min Filter

```
min_result = filter(original_matrix, "min")
min_result = np.pad(min_result, pad_width=1, mode='constant', constant_values=0)
print("\nHasil filter min:")
print(min_result)
✓ 0.0s
```

Gunakan function filter (kode diatas) dengan parameter min. Beri padding untuk mengoutputkan sesuai dengan ukuran input (karena asumsi tidak ada padding maka kasih nilai 0)

Hasil filter:

```
Hasil filter min:
[[0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 1. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]]
```